

这样例 5.10 中对 Ssex 的说明可以改写为：

```
Ssex GenderDomain
```

[例 5.15] 建立一个性别域 GenderDomain，并对其中的限制命名。

```
CREATE DOMAIN GenderDomain CHAR(6)
CONSTRAINT GD CHECK(VALUE IN ('男','女'));
```

[例 5.16] 删除域 GenderDomain 的限制条件 GD。

```
ALTER DOMAIN GenderDomain
DROP CONSTRAINT GD;
```

[例 5.17] 在域 GenderDomain 上增加性别的限制条件 GDD。

```
ALTER DOMAIN GenderDomain
ADD CONSTRAINT GDD CHECK (VALUE IN('1','0'));
```

这样，通过例 5.16 和例 5.17，就把性别的取值范围由('男','女')改为('1','0')。

SQL 标准支持数据库完整性断言 (ASSERTION) 功能。用户可以使用 DDL 中的 CREATE ASSERTION 语句，通过声明性断言 (declarative assertion) 来指定更具一般性的完整性约束，可以定义涉及多个表或聚集操作的比较复杂的完整性约束 (详细内容可扫描二维码阅读)。



扩展阅读：SQL
断言

5.7 触 发 器

触发器是用户定义在关系表上的一类由事件驱动的特殊过程。触发器类似于约束，但是比约束更加灵活，可以实施更为复杂的检查和违约操作。它不是只提供一种选择，即中止导致违约的事务/操作，因此具有更精细和更强大的数据控制能力。

触发器一经定义就被保存在数据库服务器中。任何用户对表的更新操作均由服务器自动激活相应的触发器，在关系数据库管理系统核心层进行集中的完整性控制。

触发器在 SQL 99 之后才写入 SQL 标准，但是不少关系数据库管理系统早就支持触发器，因此不同的关系数据库管理系统实现的触发器语法各不相同、互不兼容。本节的例子是按照 SQL 标准写的，不同的关系数据库管理系统产品会有差别，请读者在上机实验时注意阅读所安装系统的使用说明。

5.7.1 定义触发器

触发器又叫作事件-条件-动作规则 (event-condition-action rule, ECA rule)。当特定的系

统事件(如对一个表的更新操作,事务的结束等)发生时,对规则的条件进行检查,如果条件成立则执行规则中的动作,否则不执行该动作。规则中的动作体可以很复杂,涉及其他表和其他数据库对象,通常是一段 SQL 存储过程。

SQL 使用 CREATE TRIGGER 命令建立触发器,其一般格式如下:

```
CREATE TRIGGER <触发器名>                                /* 每当触发事件发生时,该触发器被激活 */
[BEFORE | AFTER] <触发事件> ON <表名>                    /* 指明触发器激活的时间是在执行
                                                           /* 触发事件前或后 */
REFERENCING NEW|OLD AS<变量>                             /* REFERENCING 指出引用的变量 */
FOR EACH[ROW | STATEMENT]                                /* 定义触发器的类型,指明动作体执行的频率 */
[WHEN <触发条件>]<触发动作体>                            /* 仅当触发条件为真时才执行触发动作体 */
```

下面对定义触发器的各部分语法进行详细说明。

① 创建触发器。只有表的拥有者,即创建表的用户才可以在表上创建触发器,并且一个表上只能创建一定数量的触发器。触发器的具体数量由具体的关系数据库管理系统产品在设计时确定。

② 触发器名。触发器名可以包含模式名,也可以不包含模式名。同一模式下的触发器名必须是唯一的,并且触发器名和表名必须在同一模式下。

③ 表名。触发器只能定义在基本表上,不能定义在视图上。当基本表的数据发生变化时,将激活定义在该表上相应触发事件的触发器,因此该表也称为触发器的目标表。

④ 触发事件。触发事件可以是 INSERT、DELETE 或 UPDATE,也可以是这几个事件的组合,如 INSERT OR DELETE、INSERT AND DELETE 等,还可以是 UPDATE OF <触发列, ...>,即进一步指明修改哪些列时激活触发器。

AFTER/BEFORE 是触发的时机。AFTER 表示在触发事件的操作执行之后激活触发器, BEFORE 则表示在触发事件的操作执行之前激活触发器。

⑤ 触发器类型。触发器按照所触发动作的间隔尺寸可以分为行级触发器(FOR EACH ROW)和语句级触发器(FOR EACH STATEMENT)。

如果定义的触发器为语句级触发器,那么执行了此 UPDATE 语句后触发动作体将执行一次。如果是行级触发器,那么 UPDATE 语句影响多少行,就触发多少次。

⑥ 触发条件。触发器被激活时,只有当触发条件为真时触发动作体才执行,否则触发动作体不执行。如果省略 WHEN 触发条件,则触发动作体在触发器激活后立即执行。

⑦ 触发动作体。触发动作体既可以是一个匿名 PL/SQL 过程块,也可以是对已创建存储过程的调用。如果是行级触发器,用户可以在过程体中使用 NEW 或 OLD 引用 UPDATE/INSERT 事件之后(前)的新(旧)值;如果是语句级触发器,则不能在触发动作体中使用 NEW 或 OLD 进行引用。

如果触发动作体执行失败,激活触发器的事件(即对数据库的更新操作)就会终止执行,触发器的目标表或触发器可能影响的其他对象不发生任何变化。

[例 5.18] 当对表 SC 的 Grade 属性进行修改时, 若分数增加了 10%, 则将此次操作记录到另一个表 SC_U(Sno CHAR(8), Cno CHAR(5), Oldgrade SMALLINT, Newgrade SMALLINT) 中。其中, Oldgrade 是修改前的分数, Newgrade 是修改后的分数。运行触发器之前需要创建 SC_U 表。

```
CREATE TRIGGER SC_T          /* SC_T 是触发器的名字 */
AFTER UPDATE ON SC          /* UPDATE ON SC 是触发事件, */
    /* AFTER 是触发的时机, 表示当对 SC 的 Grade 属性修改完后再触发下面的规则 */
REFERENCING
    OLD AS OldTuple
    NEW AS NewTuple
FOR EACH ROW /* 行级触发器, 即每执行一次 Grade 的更新, 下面的规则就执行一次 */
WHEN (NewTuple. Grade >= 1.1 * OldTuple. Grade)
    /* 触发条件, 只有该条件为真时才执行下面的 insert 操作 */
BEGIN
    INSERT INTO SC_U (Sno, Cno, OldGrade, NewGrade) /* 触发动作体 */
    VALUES(OldTuple. Sno, OldTuple. Cno, OldTuple. Grade, NewTuple. Grade)
END
```

在本例中 REFERENCING 指出引用的变量, 如果触发事件是 UPDATE 操作并且有 FOR EACH ROW 子句, 则可以引用的变量有 OLD 和 NEW, 分别表示修改之前的元组和修改之后的元组。若没有 FOR EACH ROW 子句, 则可以引用的变量有 OLD TABLE 和 NEW TABLE, OLD TABLE 表示表中原来的内容, NEW TABLE 表示表中变化后的部分。

[例 5.19] 将每次对表 Student 的插入操作所增加的学生个数记录到表 Student InsertLog (numbers INT) 中, 运行触发器之前需要创建此表。

```
CREATE TRIGGER Student_Count
AFTER INSERT ON Student /* 指明触发器激活的时间是在执行 INSERT 后 */
REFERENCING
    NEW TABLE AS Delta
FOR EACH STATEMENT /* 语句级触发器, 即执行完 INSERT 语句后 */
    /* 下面的触发动作体才执行一次 */
BEGIN
    INSERT INTO StudentInsertLog (Numbers)
    SELECT COUNT(*) FROM Delta
END
```

在本例中出现的“FOR EACH STATEMENT”表示触发事件 INSERT 语句执行完成后才执行一次触发器中的动作, 这种触发器叫作语句级触发器。而例 5.18 中的触发器是行级触发器。默认的触发器是语句级触发器。“Delta”是一个关系名, 其模式与 Student 相同, 包含的元组是

INSERT 语句增加的元组。

[例 5.20] 定义一个 BEFORE 行级触发器, 为教师表 Teacher 定义完整性规则: 教授的工资不得低于 4 000 元, 如果低于 4 000 元, 自动改为 4 000 元。

```
CREATE TRIGGER Update_Sal      /* 对教师表插入或更新时激活触发器 */
BEFORE UPDATE ON Teacher      /* BEFORE 触发事件 */
REFERENCING NEW AS newTuple
FOR EACH ROW                  /* 这是行级触发器 */
BEGIN                          /* 定义触发动作体, 这是一个 PL/SQL 过程块 */
    IF (newTuple.job='教授') AND (newTuple.sal < 4000)
        /* 因为是行级触发器, 可在过程体中 */
        THEN newTuple.sal := 4000; /* 使用插入或更新操作后的新值 */
    END IF;
END;                            /* 触发动作体结束 */
```

因为定义的是 BEFORE 触发器, 在插入和更新教师记录前就可以按照触发器的规则调整教授的工资, 不必等插入后再检查调整。

5.7.2 执行触发器

触发器的执行是由触发事件激活, 并由数据库服务器自动执行的。一个数据表上可能定义了多个触发器, 如多个 BEFORE 触发器、多个 AFTER 触发器等, 同一个表上的多个触发器激活时遵循如下的执行顺序:

- ① 执行该表上的 BEFORE 触发器。
- ② 激活触发器的 SQL 语句。
- ③ 执行该表上的 AFTER 触发器。

对于同一个表上的多个 BEFORE/AFTER 触发器, 遵循“谁先创建谁先执行”的原则, 即按照触发器创建的时间先后顺序执行。有些关系数据库管理系统是按照触发器名称的字母排序顺序执行触发器。

5.7.3 删除触发器

删除触发器的 SQL 语句格式如下:

DROP TRIGGER <触发器名> ON <表名>;

触发器必须是一个已经创建的触发器, 并且只能由具有相应权限的用户删除。

触发器是一种功能强大的工具, 但在使用时要慎重, 因为在每次访问一个表时都可能触发一个触发器, 这样会影响系统性能。

本章小结

本章系统讲解了数据库完整性的概念,介绍了使用 SQL 语句实现数据库完整性定义的方法。关系数据库管理系统将负责检查完整性约束,当发现违约时进行处理。对于违反参照完整性的操作,本书讲解了不同的处理策略。用户要根据应用语义来定义合适的处理策略,以保证数据库的正确性。

本章还介绍了实现数据库完整性的方法之一——触发器。

触发器与各种完整性约束方法相比,其违约处理的功能更加灵活,更加强大。当违约时,触发器的动作体可以是一段 SQL 存储过程或 PL/SQL 过程块。触发器不仅可以用于数据库完整性检查,也可以用来实现数据库系统的其他功能,包括数据库安全性、应用系统的一些业务流程和控制流程等。不过也要注意,一个触发器的动作可能激活另一个触发器,最坏的情况是导致激活一个触发链,从而有可能造成难以预见的错误。

习题 5

1. 什么是数据库的完整性?
2. 数据库的完整性概念与数据库的安全性概念有什么区别和联系?
3. 什么是数据库的完整性约束?
4. 关系数据库管理系统的完整性控制机制应具备哪三方面的功能?
5. 关系数据库管理系统在实现参照完整性时需要考虑哪些方面?
6. 假设下面两个关系模式:

职工(职工号,姓名,出生日期,职务,工资,部门号),其中职工号为主码;
部门(部门号,名称,经理姓名,电话),其中部门号为主码。

用 SQL 定义这两个关系模式,要求在模式中完成以下完整性约束的定义:

- ① 定义每个模式的主码。
- ② 定义参照完整性约束。
- ③ 定义职工年龄不超过 65 岁。

7. 在关系数据库管理系统中,当操作违反实体完整性、参照完整性和用户定义的完整性约束时,一般是如何分别进行处理的?

* 8. 某单位想举行一个小型联谊会,关系 Male 记录注册的男宾信息,关系 Female 记录注册的女宾信息。建立一个断言,将来宾的人数限制在 50 人以内(提示:先创建关系 Female 和关系 Male)。

第5章实验 完整性控制

实验 5.1 实体完整性实验。掌握实体完整性的定义和维护方法。定义实体完整性,删除实体完整性。能够写出两种方式定义实体完整性的 SQL 语句:创建表时定义实体完整性、创建表后使用 ALTER TABLE 语句定义实体完整性。设计 SQL 语句验证完整性约束是否起作用。

实验 5.2 参照完整性实验。掌握参照完整性的定义和维护方法。定义参照完整性,定义参照完整性的违

约处理, 删除参照完整性。能够写出两种方式定义参照完整性的 SQL 语句: 创建表时定义参照完整性、创建表后使用 ALTER TABLE 语句定义参照完整性。

实验 5.3 用户自定义完整性实验。掌握用户自定义完整性的定义和维护方法。针对具体应用语义, 选择 NULL/NOT NULL、DEFAULT、UNIQUE、CHECK 等定义属性上的约束条件。

实验 5.4 触发器实验。掌握数据库触发器的设计和使用方法。定义 BEFORE 触发器和 AFTER 触发器、行级触发器或语句级触发器。能够理解不同类型触发器的作用和执行原理, 设计数据操纵语言(DML)语句验证触发器的有效性。

参考文献 5

[1] HAMMER M M, MCLEOD D J. Semantic integrity in a relational data base system[C]. Proceedings of the 1st International Conference on Very Large Data Bases, 1975: 25-47.

文献[1]讨论数据库管理系统中语义完整性的概念。

[2] CODD E F. Extending the data base relational model to capture more meaning[J]. ACM Transactions on Database Systems, 1979, 4(4): 397-434.

文献[2]扩展了关系模型的语义表达能力。

[3] STONEBRAKER M. Implementation of integrity constraints and views by query modification[C]. Proceedings of International Conference on ACM SIGMOD, 1975.

文献[3]讨论 INGRES 数据库管理系统的完整性约束机制。

[4] CHAMBERLIN D D, ASTRAHAN M M, ESWARAN K P, et al. Sequel 2: a unified approach to data definition, manipulation, and control[J]. IBM Journal of Research and Development, 1976, 20(6): 560-575.

文献[4]提出了 SYSTEM R 中 SQL 的 Integrity Assert(完整性断言)和触发器语句。

[5] BERNSTEIN P A, BLAUSTEIN B T, CLARKE E M. Fast maintenance of semantic integrity assertions using redundant aggregate data[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 126-136.

[6] HSU A, IMIELINSKI T. Integrity checking for multiple updates[J]. ACM SIGMOD Record, 1985, 14(4): 152-168.

本章知识点讲解微视频:



模型固有的完整性



用户定义的完整性



5-3 通用工具——

触发器

第二篇

设计与应用开发篇

本篇讲解在开发应用系统时如何设计数据库，以及如何基于已经选定的数据库管理系统进行编程。

本篇包括三章。

第6章关系数据理论，详细讲解关系规范化理论，它既是关系数据库的重要理论基础，也是数据库设计的有力工具。规范化理论为数据库设计提供了理论指南和工具。

第7章数据库设计，讨论数据库设计的方法和步骤，详细讲解数据库设计各阶段的目标、方法和应注意的问题；重点讲解概念结构和逻辑结构的设计。以“高校本科教务管理”信息系统为示例，抽象出“学生选课管理”“学生学籍管理”“教师教学管理”三个应用场景，阐述如何在理论的指导下完成 E-R 图的设计、集成和优化。

第8章数据库编程，从两方面讲解数据库编程技术和方法：一是使用扩展的 SQL 功能，包括引入新的 SQL 子句、新的数据库内置函数、PL/SQL 与存储过程/自定义函数；二是在高级语言程序中使用 SQL，实现高级语言与数据库之间的交互，开发应用需求中复杂业务逻辑的技术和方法。本章还介绍了 JDBC 编程的原理、方法和步骤。

关系数据理论的主要内容是关系规范化理论,用来指导数据库逻辑设计,具体包括两部分:一是给出判断数据库逻辑设计“好坏程度”的准则;二是如果逻辑设计中存在“不好”的关系模式,如何将其修改为“好”的关系模式。

本章导读

在关系数据库模式设计中,我们面临两个问题,第一,什么是一个“好”的关系数据库模式,其标准是什么。如果没有判断的标准,就很难对一个具体的设计进行评价。第二,能否开发数据库设计辅助工具来减轻设计者的负担。本章6.1和6.2节介绍关系规范化理论,用于回答第一个问题;6.3和6.4节介绍函数依赖公理系统以及模式分解算法,探索自动进行模式分解的方法,用于回答第二个问题。

本章首先从数据库逻辑设计中如何构造一个好的数据库模式这一问题出发,阐明关系规范化理论研究的实际背景,然后讨论两类最重要的数据依赖——函数依赖和多值依赖。读者要牢固掌握数据依赖的基本概念,如函数依赖、部分函数依赖、完全函数依赖、传递函数依赖等,掌握1NF到4NF的概念、规范化的含义和作用;能够了解各种范式以及在插入、删除和更新数据时可能出现的问题,并直观地描述解决办法。

本章进一步讨论函数依赖的推理规则,介绍函数依赖公理系统,讲解保持函数依赖的模式等价准则及相应的模式分解算法。要掌握这些算法并在数据库模式设计中应用。6.5节是选读内容,介绍无损连接的模式分解概念及相应算法,并解释其与保持函数依赖的模式分解之间的关系。

读者学习本章后应能够根据应用语义完整地写出关系模式的数据依赖集合,并能根据数据依赖分析某一个关系模式属于第几范式,能够运用介绍的有关算法辅助设计和优化数据库模式。

6.1 问题的提出

前面已经讨论了数据库系统的一般概念,介绍了关系数据库的基本概念、关系模型的三要